

CS253 Assignment 1: Report

Memory-Efficient Versioned File Indexer in C++

Student Name:	Kamjula Sumanth Reddy
Roll Number:	240510
Course:	CS253 - Software Development and DevOps
Instructor:	Prof. Pritam Choudhury
Date of Submission:	March 6, 2026

1. Executive Summary

This report documents the design, implementation, and performance characteristics of a high-efficiency versioned text file indexer in C++. The system reads massive text logs incrementally using fixed-size buffers, processes the tokens in an online fashion, and keeps memory consumption bounded. Additionally, it models different document inputs as separate version mappings to perform word count frequency analysis, differential frequency comparisons, and top-K analytics.

2. Architectural & Class Design

The application is structured into clearly separated classes with singular responsibilities to enforce solid OOP design:

- **Tokenizer:** Handles character-level extraction of lowercased alphanumeric words, including state preservation for broken words.
- **BufferedReader:** Manages direct low-level block file streaming with standard C++ file streams.
- **VersionedIndexer:** Encapsulates version-controlled mapping maps (a hash map of hash maps) of terms and frequencies.
- **Query (Abstract Base Class):** Mandates polymorphic execution behavior. Subclassed by *WordQuery*, *DiffQuery*, and *TopKQuery* to allow seamless runtime query dispatch.

3. Core C++ Requirements Demonstrated

The solution is written in modern standard C++ (C++17) and demonstrates the following mandatory requirements:

- 1. Inheritance & Runtime Polymorphism:** The base class *Query* defines *virtual void execute() = 0*. Derived query subclasses implement customized analytic queries, executed polymorphically via virtual dispatch.
- 2. Function Overloading:** The *VersionedIndexer* implements overloaded methods: *void addWord(version, word)* (increments frequency by 1) and *void addWord(version, word, count)* (increments frequency by a specified count).
- 3. Generic Templates:** The generic *QueryResult<T>* class template prints standard results. An explicit template specialization *QueryResult<vector<pair<string, int>>>* handles specific formatting for top-K listings.
- 4. Structured Exception Handling:** File errors, parameter checking, and buffer constraint violations are guarded with standard *try-catch-throw* blocks.

4. Boundary Token Handling Algorithm

When streaming file chunks using a fixed-size buffer (e.g., 512 KB), tokens can be split across chunk boundaries. A word like 'error' might be split such that 'er' is at the end of Chunk N, and 'ror' is at the beginning of Chunk N+1.

Boundary Handling Mechanism:

1. The tokenizer maintains a *leftover* string buffer.
2. For each chunk read, the previous *leftover* is prepended to the new chunk before processing.
3. If the combined buffer's last character is alphanumeric, the last parsed word might be incomplete. It is popped from the completed words list and saved in *leftover*.
4. At EOF, the *leftover* is processed as a complete word since no more chunks follow.

5. Memory Boundedness Analysis

To process extremely large log files, memory usage must be strictly controlled:

Data Component	Memory Growth Model	Dependencies
File Read Buffer	$O(1)$ - Bounded and Constant	Set by user: 256 KB to 1024 KB
Token List (Chunk)	$O(\text{Buffer Size})$ - Bounded	Max words in a single buffer
Indexer Map	$O(U)$ - Unique Words Only	Proportional to unique vocabulary size
Leftover Buffer	$O(L)$ - Longest Word Length	Length of the longest single token (~100 B)

Since the file is never read entirely into memory, memory consumption does not scale with the size of the file. Even on a multi-gigabyte log file, the RSS (Resident Set Size) remains low and constant throughout the program execution, except for the map containing unique word counts, which grows slowly based on Heaps' Law ($V = k N^{\eta}$ where $\eta \approx 0.5$).

6. Verification & Experimental Results

We executed the compiled executable `analyzer` on the 113 MB `test\_logs.txt` and `verbose\_logs.txt` files under different constraints. The following console runs verify correctness:

Word Count Query Run (Buffer: 256 KB):

```
./analyzer --file test_logs.txt --version v1 --buffer 256 --query word --word error
Version: v1
Word Count (error): 605079
Allocated Buffer Size: 256 KB
Execution Time: 0.76 seconds
```

Top-10 Words Query Run (Buffer: 512 KB):

```
./analyzer --file test_logs.txt --version v1 --buffer 512 --query top --top 10
Version: v1
Top 10 words:
devops 1209558
debug 605150
error 605079
info 604266
warning 604149
orderservice 484437
paymentservice 484078
authservice 483842
searchservice 483162
userservice 483125
Allocated Buffer Size: 512 KB
Execution Time: 0.73 seconds
```

Difference Query Run (Buffer: 1024 KB):

```
./analyzer --file1 test_logs.txt --version1 v1 --file2 verbose_logs.txt --version2
v2 --buffer 1024 --query diff --word error
Versions: v1 vs v2
Difference for error: -495377
Allocated Buffer Size: 1024 KB
Execution Time: 1.56 seconds
```

Conclusion:

The system has been verified to be robust, memory-efficient, and correct. All boundary edge cases are handled as expected. The memory footprint remains stable and bounded, showing standard compliance with the constraints.